

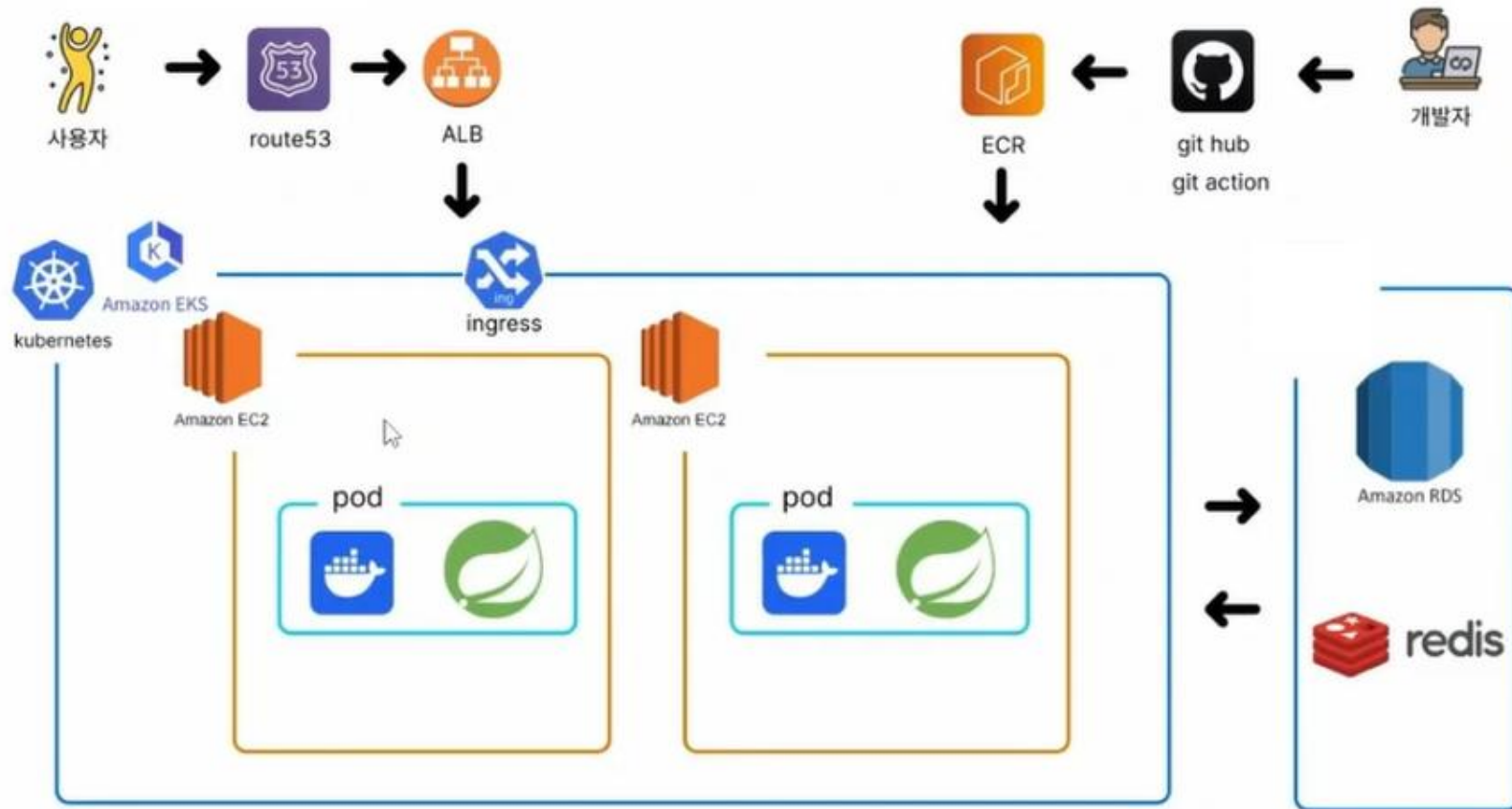
AWS 기본

inky4832@daum.net

15. CI/CD 배포

15.1 아키텍처 미리 보기

실습 아키텍처



15.2 데이터베이스 인프라 구축

15.2 RDS 생성하기

1. RDS 용 보안그룹 생성하기

보안 그룹 생성 정보

보안 그룹은 인바운드 및 아웃바운드 트래픽을 관리하는 인스턴스의 가상 방화벽 역할을 합니다. 새 보안 그룹을 생성하려면 아래의 필드를 작성하십시오.

기본 세부 정보

보안 그룹 이름 정보

my-db

영문, 숫자, 하이픈만 입력 가능합니다.

설명 정보

VPC 정보

인바운드 규칙 정보

유형 정보

프로토콜 정보

포트 범위 정보

소스 정보

설명 - 선택 사항 정보

15.2 RDS 생성하기

2. RDS 생성하기

데이터베이스 (0) 그룹 리소스 ⚡ 🔄 수정 작업 ▼ **데이터베이스 생성 ▲**

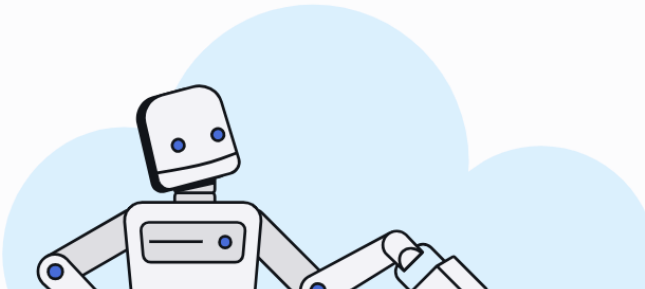
🔍 데이터베이스을(를) 기준으로 필터링

DB 식별자 ▲ | 상태 ▼ | 역할 ▼ | 엔진 | S3에서 복원 |

빠른 구성

전체 구성

S3에서 복원



15.2 RDS 생성하기

2. RDS 생성하기

[Aurora and RDS](#) > [데이터베이스](#) > 데이터베이스 생성

데이터베이스 생성 정보

엔진 옵션

엔진 유형 정보

☐ Aurora (MySQL Compatible)



☐ Aurora (PostgreSQL Compatible)



☒ MySQL



☐ PostgreSQL



☐ MariaDB



☐ Oracle

ORACLE

☐ Microsoft SQL Server



☐ IBM Db2

IBM Db2

데이터베이스 생성 방식 선택

☒ 전체 구성

가용성, 보안, 백업 및 유지 관리에 대한 옵션을 포함하여 모든 구성 옵션을 설정합니다.

☐ 손쉬운 생성

관장 모범 사례 구성을 사용합니다. 일부 구성 옵션은 데이터베이스를 생성한 후 변경할 수 있습니다.

템플릿

해당 사용 사례를 충족하는 샘플 템플릿을 선택하세요.

☐ 프로덕션

고가용성 및 빠르고 일관된 성능을 위해 기본값을 사용하세요.

☐ 개발/테스트

이 인스턴스는 프로덕션 환경 외부에서 개발 용도로 마련되었습니다.

☒ 샌드박스

새로운 애플리케이션을 개발하거나, 기존 애플리케이션을 테스트하거나 Amazon RDS에서 실무 경험을 쌓을 수 있습니다.

15.2 RDS 생성하기

DB 인스턴스 식별자

DB 인스턴스 이름을 입력하세요. 이름은 현재 AWS 리전에서 AWS 계정이 소유하는 모든 DB 인스턴스에 대해 고유해야 합니다.

my-database

DB 인스턴스 식별자는 대소문자를 구분하지 않지만 'mydbinstance'와 같이 모두 소문자로 저장됩니다. 제약: 1~63자의 영숫자 또는 하이픈으로 구성되어야 합니다. 첫 번째 문자는 글자여야 합니다. 하이픈 2개가 연속될 수 없습니다. 하이픈으로 끝날 수 없습니다.

마스터 사용자 이름 [정보](#)

DB 인스턴스의 마스터 사용자에 로그인 ID를 입력하세요.

admin

1~16자의 영숫자. 첫 번째 문자는 글자여야 합니다.

자격 증명 관리

AWS Secrets Manager를 사용하거나 마스터 사용자 자격 증명을 관리할 수 있습니다.

☐

AWS Secrets Manager에서 관리 - **가장 뛰어난 안정성**

RDS의 자동으로 암호를 생성하고 AWS Secrets Manager를 사용하여 전체 수명 주기 동안 암호를 관리합니다.

☒

자체 관리

사용자가 암호를 생성하거나 RDS에서 암호를 생성하고 사용자가 관리할 수 있습니다.

☐

암호 자동 생성

Amazon RDS에서 자동으로 암호를 생성하거나 사용자가 직접 암호를 지정할 수 있습니다.

마스터 암호 [정보](#)

password

Password strength

Very weak

최소 제약 조건: 8자 이상의 인쇄 가능한 ASCII 문자를 사용합니다. / ' " @ 기호는 포함할 수 없습니다.

마스터 암호 확인 [정보](#)

15.2 RDS 생성하기

인스턴스 구성

아래의 DB 인스턴스 구성 옵션은 위에서 선택한 엔진에서 지원하는 옵션으로 제한됩니다.

DB 인스턴스 클래스 [정보](#)

▼ 필터 숨기기

☐ Amazon RDS 최적화된 쓰기를 지원하는 인스턴스 클래스 표시 [정보](#)
Amazon RDS 최적화된 쓰기는 추가 비용 없이 쓰기 처리량(throughput)을 최대 2배 늘립니다.

☐ 이전 세대 클래스 포함

☐ 스탠다드 클래스(m 클래스 포함)

☐ 메모리 최적화 클래스(r 및 x 클래스 포함)

☒ 버스터블 클래스(t 클래스 포함)

인스턴스 유형

db.t4g.micro
2 vCPUs 1 GiB RAM EBS 대역폭: 최대 2,085Mbps 네트워크: 최대 5Gbps

스토리지

스토리지 유형 [정보](#)

이제 프로비저닝된 IOPS SSD(io2) 스토리지 볼륨을 사용할 수 있습니다.

범용 SSD(gp2)
볼륨 크기에 따라 기준 성능 결정

할당된 스토리지 [정보](#)

20

할당된 스토리지 값은 20GiB~6,144GiB여야 합니다

▶ 추가 스토리지 구성

GiB

15.2 RDS 생성하기

연결 정보

컴퓨팅 리소스

이 데이터베이스의 컴퓨팅 리소스에 대한 연결을 설정할지를 선택합니다. 연결을 설정하면 컴퓨팅 리소스가 이 데이터베이스에 연결할 수 있도록 연결 설정이 자동으로 변경됩니다.

☒ EC2 컴퓨팅 리소스에 연결 안 함

이 데이터베이스의 컴퓨팅 리소스에 대한 연결을 설정하지 않습니다. 나중에 컴퓨팅 리소스에 대한 연결을 수동으로 설정할 수 있습니다.

☐ EC2 컴퓨팅 리소스에 연결

이 데이터베이스의 EC2 컴퓨팅 리소스에 대한 연결을 설정합니다.

Virtual Private Cloud(VPC) 정보

VPC를 선택합니다. VPC는 이 DB 인스턴스의 가상 네트워킹 환경을 정의합니다.

Default VPC (vpc-0beacd6fe172a4e5f)

4 서브넷, 4 가용 영역



데이터베이스가 생성된 후에는 VPC를 변경할 수 없습니다. 해당 DB 서브넷 그룹이 있는 VPC만 나열됩니다.

i 데이터베이스를 생성한 후에는 VPC를 변경할 수 없습니다.

DB 서브넷 그룹 정보

DB 서브넷 그룹을 선택합니다. DB 서브넷 그룹은 선택한 VPC에서 DB 인스턴스가 어떤 서브넷과 IP 범위를 사용할 수 있는지를 정의합니다.

default-vpc-0beacd6fe172a4e5f

4 서브넷, 4 가용 영역

퍼블릭 액세스 정보

☒ 예

RDS는 데이터베이스에 퍼블릭 IP 주소를 할당합니다. VPC 외부의 Amazon EC2 인스턴스 및 다른 리소스가 데이터베이스에 연결할 수 있습니다. VPC 내부의 리소스도 데이터베이스에 연결할 수 있습니다. 데이터베이스에 연결할 수 있는 리소스

☐ 아니요

RDS는 퍼블릭 IP 주소를 데이터베이스에 할당하지 않습니다. VPC 내부의 Amazon EC2 인스턴스 및 다른 리소스만 데이터베이스에 연결할 수 있습니다. 데이터베이스에 연결할 수 있는 리소스를 지정하는 VPC 보안 그룹을 하나 이상 선택합니다

15.2 RDS 생성하기

VPC 보안 그룹(방화벽) 정보

데이터베이스에 대한 액세스를 허용할 VPC 보안 그룹을 하나 이상 선택합니다. 보안 그룹 규칙이 적절한 수신 트래픽을 허용하는지 확인합니다.

☒ 기존 항목 선택
기존 VPC 보안 그룹 선택

☐ 새로 생성
새 VPC 보안 그룹 생성

기존 VPC 보안 그룹

하나 이상의 옵션 선택

my-db X

가용 영역 정보

기본 설정 없음

RDS 프록시

RDS 프록시는 애플리케이션 확장성, 복원력 및 보안을 개선하는 완전관리형 고가용성 데이터베이스 프록시입니다.

☐ RDS 프록시 생성 정보

RDS는 프록시에 대한 IAM 역할과 Secrets Manager 보안 암호를 자동으로 생성합니다. RDS 프록시에 대한 추가 비용이 있습니다. 자세한 내용은 다음을 참조하세요. [Amazon RDS 프록시 요금](#).

인증 기관 - 선택 사항 정보

서버 인증서를 사용하면 Amazon 데이터베이스에 대한 연결이 이루어지고 있는지 검증하여 추가 보안 계층을 제공합니다. 프로비저닝하는 모든 데이터베이스에 자동으로 설치되는 서버 인증서를 확인하여 이를 수행합니다.

rds-ca-rsa2048-g1 (기본값)

만료: May 21, 2061

인증 기관을 선택하지 않으면 RDS에서 대신 인증 기관을 선택합니다.

▼ 추가 구성

데이터베이스 포트 정보

데이터베이스가 애플리케이션 연결에 사용할 TCP/IP 포트입니다.

3306

15.2 RDS 생성하기

▼ 추가 구성

데이터베이스 옵션, 암호화 켜짐, 백업 켜짐, 역추적 꺼짐, 유지 관리, CloudWatch Logs, 삭제 방지 꺼짐.

데이터베이스 옵션

초기 데이터베이스 이름 [정보](#)

testdb

testdb

데이터베이스 이름을 지정하지 않으면 Amazon RDS에서 데이터베이스를 생성하지 않습니다.

DB 파라미터 그룹 [정보](#)

default.mysql8.4

옵션 그룹 [정보](#)

default:mysql-8-4

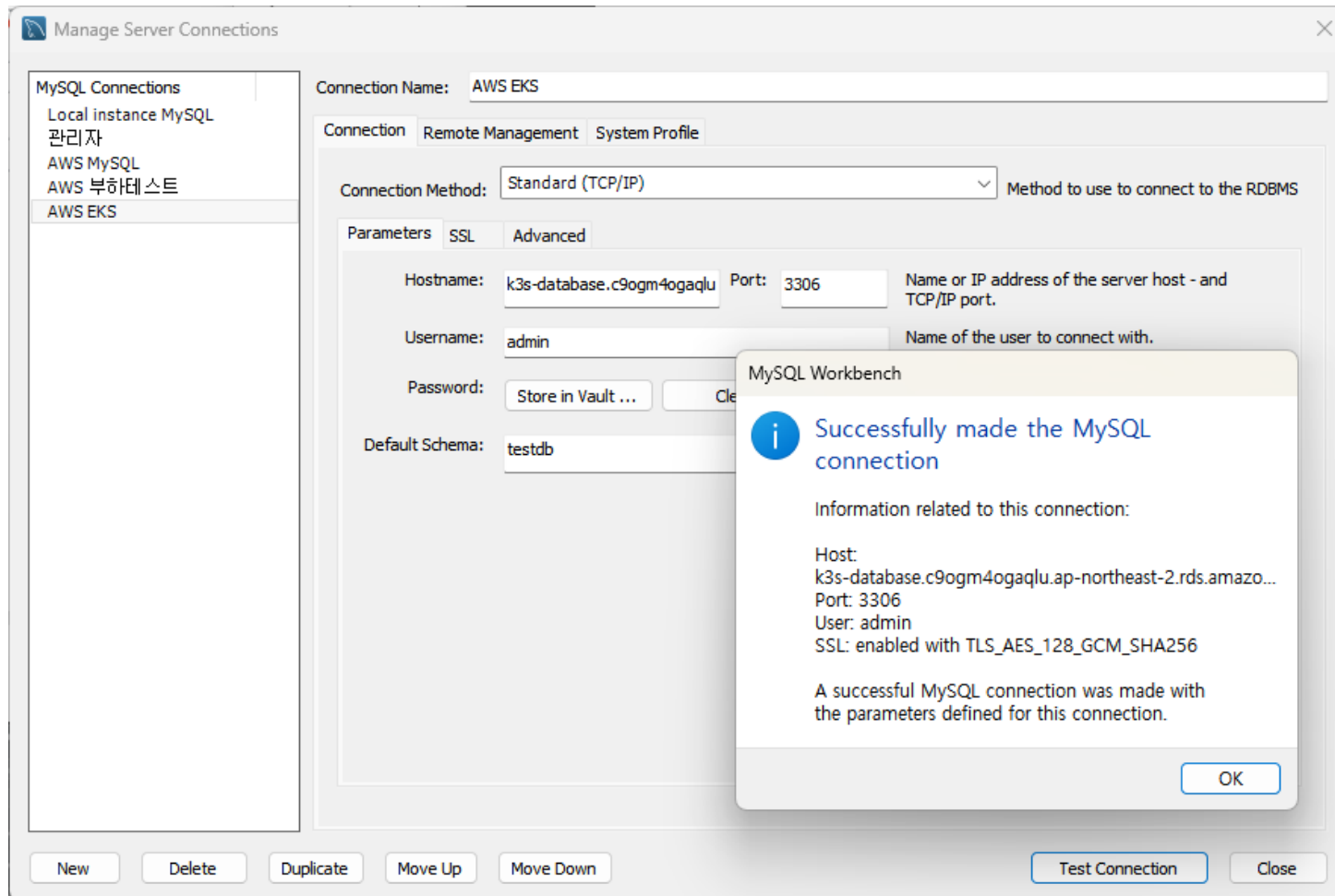
☒ 암호화 활성화

지정된 인스턴스를 암호화하려면 이 옵션을 선택합니다. AWS Key Management Service 콘솔을 사용하여 마스터 키 ID와 별칭이 생성된 후 해당 항목이 목록에 표시됩니다.

12

15.2 RDS 생성하기

3. 접속 확인



15.2 RDS 생성하기

18. Redis 생성 (redis_AWS.yaml)

```
kubectl apply -f redis_AWS.yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis
spec:
  replicas: 1
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
        - name: redis
          image: redis:7-alpine
          ports:
            - containerPort: 6379
```

```
---
apiVersion: v1
kind: Service
metadata:
  name: redis
spec:
  ports:
    - port: 6379
      targetPort: 6379
  selector:
    app: redis
```

15.2 RDS 생성하기

19. Secret 생성 (db_secret_AWS.yaml)

```
kubectl apply -f db_secret_AWS.yaml
```

```
apiVersion: v1  
kind: Secret  
type: Opaque # 범용
```

```
# Secret 기본 정보
```

```
metadata:  
  name: my-app-secrets # Secret 이름
```

```
# Key, Value 형식으로 값 저장
```

```
stringData:  
  DB_HOST: my-database.cnqmcq6uwqa3.ap-northeast-2.rds.amazonaws.com  
  DB_PORT: "3306"  
  DB_NAME: testdb  
  DB_USERNAME: admin  
  DB_PASSWORD: password
```


15.3 ECR 인프라 구축

15.3 ECR 인프라 구축

1. 프라이빗 저장소 생성

컨테이너

Amazon Elastic Container Registry

컨테이너 소프트웨어를 공개적으로 또는 비공개로 공유 및 배포합니다.

Amazon Elastic Container Registry(ECR)는 완전관리형 컨테이너 레지스트리로, 이미지와 아티팩트를 어디서나 쉽게 보관, 관리, 공유 및 배포하도록 지원합니다.

리포지토리 생성

생성

요금(미국)

퍼블릭 또는 프라이빗 리포지토리에 저장한 데이터 및 인터넷으로 전송된 데이터 양에 대해서만 요금을 지불합니다.

[ECR 요금](#)

VPC 인터페이스 엔드포인트를 사용하여 ECR 프라이빗 리포지토리에 대한 프라이빗 연결을 활성화하고 비용을 절감합니다.

[자세히 알아보기](#)
[VPC 인터페이스 엔드포인트 배포](#)

시작하기

[Amazon ECR은 무엇입니까?](#)

[ECR 시작하기](#)

작동 방식

```

graph LR
    A[Write code  
Write and package code as a Docker image] --> B[Amazon ECR]
    B --> C[Compress, encrypt, and control access to images]
    C --> D[Version, tag, and manage image lifecycles]
    D --> E[Run containers  
Pull images and run containers anywhere]
    E --> F[Amazon ECS]
    E --> G[Amazon EKS]
    E --> H[Amazon Cloud]
    E --> I[On premises]
  
```

15.3 ECR 인프라 구축

Amazon ECR > 프라이빗 레지스트리 > 리포지토리 > 프라이빗 리포지토리 생성

①

프라이빗 리포지토리 생성

일반 설정

리포지토리 이름

간결한 이름을 입력합니다. 리포지토리는 네임스페이스를 지원하며, 이를 사용하여 유사한 리포지토리를 그룹화할 수 있습니다.

727646470302.dkr.ecr.ap-northeast-2.amazonaws.com/my-ecr **my-ecr**

6로 최대 문자 수 256(최소 2)을(를) 초과했습니다. 이름은 문자로 시작해야 하며 소문자, 숫자 및 특수 문자(._/만 포함할 수 있습니다.

이미지 태그 설정 정보

이미지 태그 변경 가능성

태그 변경 가능성 설정을 선택합니다.

☒ Mutable

이미지 태그를 덮어쓸 수 있습니다.

☐ Immutable

이미지 태그를 덮어쓸 수 없습니다.

변경 가능한 태그 제외

이러한 필터와 일치하는 태그는 변경할 수 없습니다(덮어쓰기 불가능). 0개 이상의 이미지 태그 문자와 일치시키려면 와일드카드 (*)를 사용합니다.

필터 추가

필터에는 문자, 숫자, 특수 문자(._*)만 포함해야 합니다. 각 필터는 128자, 와일드카드(*) 2개로 제한되며 제외 목록에 최대 5개의 필터를 추가할 수 있습니다.

암호화 설정 정보

⚠ 리포지토리 생성 후에는 리포지토리의 암호화 설정을 변경할 수 없습니다.

암호화 구성

기본적으로 리포지토리는 업계 표준 Advanced Encryption Standard (AES) 암호화를 사용합니다. 선택적으로 AWS Key Management Service (KMS)에 저장된 키를 사용하여 리포지토리의 이미지를 암호화하도록 선택할 수 있습니다.

☒ AES-256

업계 표준 Advanced Encryption Standard (AES) 암호화

☐ AWS KMS

AWS Key Management Service (KMS)

▶ 이미지 스캔 설정 - deprecated

취소

생성

15.3 ECR 인프라 구축

🟢 프라이빗 리포지토리를 생성함, my-ecr

프라이빗 리포지토리 (1)

🔍 리포지토리 하위 문자열로 검색

리포지토리 이름	리포지토리 URI 복사됨	생성 날짜	태그 변경 불가능
my-ecr	727646470302.dkr.ecr.ap-north-east-2.amazonaws.com/my-ecr	2026년 7월 12일, 15:34:19 (UTC+09)	변경 가능

my-ecr

요약 | **이미지** | 수명 주기 정책 | 권한 | 리포지토리 태그

이미지 정보

🔍 활성 이미지 필터링

☐ 이미지 태그 ▼
 유형
 생성 날짜: ▼
 이미지 크기(MB) ▼
 이미지 다이제스트
 마지막으로 풀링 된 시간: ▼

활성 이미지 없음

🔄 삭제 📄 URI 복사 세부 정보 스캔 **푸시 명령 보기**

15.3 ECR 인프라 구축

my-ecr에 대한 푸시 명령



macOS / Linux

Windows

최신 버전의 AWS CLI 및 Docker이(가) 설치되어 있는지 확인합니다. 자세한 내용은 [Amazon ECR 시작하기](#) 을(를) 참조하세요.

다음 단계를 사용하여 이미지를 인증하고 리포지토리에 푸시합니다. Amazon ECR 자격 증명 헬퍼를 비롯한 추가 레지스트리 인증 방법은 [레지스트리 인증](#) 을(를) 참조하십시오.

1. 인증 토큰을 검색하고 레지스트리에 대해 Docker 클라이언트를 인증합니다. 다음 AWS CLI을(를) 사용하세요.

```
aws ecr get-login-password --region ap-northeast-2 | docker login --username AWS --password-stdin 727646470302.dkr.ecr.ap-northeast-2.amazonaws.com
```

참고: AWS CLI을(를) 사용하는 중 오류가 발생하면 최신 버전의 AWS CLI 및 Docker가 설치되어 있는지 확인하세요.

2. 다음 명령을 사용하여 도커 이미지를 빌드합니다. 도커 파일을 처음부터 새로 빌드하는 방법에 대한 자세한 내용은 [여기](#) 지침을 참조하십시오. 이미지를 이미 빌드한 경우에는 이 단계를 건너뛸 수 있습니다.

```
docker build -t my-ecr .
```

3. 빌드가 완료되면 이미지에 태그를 지정하여 이 리포지토리에 푸시할 수 있습니다.

```
docker tag my-ecr:latest 727646470302.dkr.ecr.ap-northeast-2.amazonaws.com/my-ecr:latest
```

4. 다음 명령을 실행하여 이 이미지를 새로 생성한 AWS 리포지토리로 푸시합니다.

```
docker push 727646470302.dkr.ecr.ap-northeast-2.amazonaws.com/my-ecr:latest
```

닫기

15.3 ECR 인프라 구축

21. 로컬에서 Spring Boot 프로젝트를 Docker 이미지로 빌드한 후에 ECR로 Push 하기

```
c:\app-monolithic> aws ecr get-login-password --region ap-northeast-2 | docker  
login --username AWS --password-stdin 727646470302.dkr.ecr.ap-northeast-  
2.amazonaws.com
```

반드시 다음 명령어 실행하기전에 Docker desktop 먼저 실행하자

```
c:\app-monolithic > docker build -t my-ecr .
```

주의: 태그명에 latest 로 하지 말고 구체적인 버전명을 작성한다.

```
c:\app-monolithic > docker tag my-ecr:latest 727646470302.dkr.ecr.ap-  
northeast-2.amazonaws.com/my-ecr:1.0
```

```
c:\app-monolithic > docker push 727646470302.dkr.ecr.ap-northeast-  
2.amazonaws.com/my-ecr:1.0
```

my-ecr

요약

이미지

수명 주기 정책

권한

리포지토리 태그

이미지 (3) 정보

삭제

URI 복사

세부 정보

스캔

푸시 명령 보기

Q

활성 이미지 필터링

☐	이미지 태그 ▼	유형	생성 날짜: ▼	이미지 크기(MB) ▼	이미지 다이제스트	마지막으로 풀링 된 시간: ▼
☐	1.0	Image Index	2026년 7월 12일, 15:47:37 (UTC+09)	140.66	sha256:e8c...	-
☐	-	Image	2026년 7월 12일, 15:47:37 (UTC+09)	140.66	sha256:13d...	-
☐	-	Image	2026년 7월 12일, 15:47:37 (UTC+09)	0.00	sha256:083...	-

15.3 ECR 인프라 구축

[리포지토리](#) > [my-ecr](#) > sha256:5c9935dc4747464b990ad3e88b69686ba468d29bb3357daab93921...



mage Index

[아카이브](#)

세부 정보

이미지 태그
1.0

URI

 727646470302.dkr.ecr.ap-northeast-2.amazonaws.com/my-ecr:1.0

deploy 위한 yaml파일에서
Docker 이미지 경로임

다이제스트

 sha256:5c9935dc4747464b990ad3e88b69686ba468d29bb3357daab93921970024edce

일반 정보

아티팩트 유형
[Image Index](#)

리포지토리
my-ecr

푸시 위치
2026년 7월 12일, 22:18:32 (UTC+09)

마지막으로 기록된 풀타임
-

크기(MB)
140.66

이미지 상태
 **활성**

15.4 EKS

15.4 EKS 생성하기

1. 서브넷 자동 할당 IP 설정 활성화

VPC > 서브넷 > subnet-0e3a746dc23c879cb > 서브넷 설정 편집

서브넷 설정 편집 정보

서브넷
서브넷 ID
subnet-0e3a746dc23c879cb

이름
demo-vpc-inky4832-public-subnet-a

자동 할당 IP 설정 정보
AWS가 이 서브넷에 있는 인스턴스의 새 기본 네트워크 인터페이스에 퍼블릭 IPv4 또는 IPv6 주소를 자동으로 할당할 수 있도록 합니다.
☒ 퍼블릭 IPv4 주소 자동 할당 활성화 정보
☐ 고객 소유 IPv4 주소 자동 할당 활성화 정보
고객 소유 풀을 찾을 수 없어 옵션이 비활성화되었습니다.

리소스 기반 이름(RBN) 설정 정보
이 서브넷의 EC2 인스턴스에 대한 호스트 이름 유형과 선택적 RBN DNS 쿼리 설정을 지정합니다.
☐ 시작 시 리소스 이름 DNS A 레코드 활성화 정보
☐ 시작 시 리소스 이름 DNS AAAA 레코드 활성화 정보
호스트 이름 유형 정보
☐ 리소스 이름
☒ IP 이름

DNS64 설정
Amazon VPC의 IPv6 전용 서비스가 IPv4 전용 서비스 및 네트워크와 통신할 수 있도록 DNS64를 활성화합니다.
☐ DNS64 활성화 정보

취소 저장

15.4 EKS 생성하기

2. EKS 클러스터의 IAM 역할 생성하기

EC2 VPC Aurora and RDS Route 53 CloudFront Certificate Manager S3 IAM CloudWatch Elastic Container Registry

IAM > 역할

Identity and Access Management(IAM)

IAM 검색

대시보드

▼ 액세스 관리

역할

정책

IAM 사용자

역할 (25) 정보

IAM 역할은 단기간 동안 유효한 자격 증명을 가진 특정 권한이 있는 자격 증명입니다. 신뢰할 수 있는 엔티티가 역할을 맡을 수 있습니다.

검색

< 1 2 > ⚙

역할 이름	신뢰할 수 있는 개체
☐ Amazon_EventBridge_Scheduler_LAMBDA_689de297a9	AWS 서비스: scheduler
☐ Amazon_EventBridge_Scheduler_LAMBDA_8bdbf5deab	AWS 서비스: scheduler
☐ AWSServiceRoleForAutoScaling	AWS 서비스: autoscaling (선택)

15.4 EKS 생성하기

신뢰할 수 있는 엔터티 선택 정보

신뢰할 수 있는 엔터티 유형

☒ AWS 서비스

EC2, Lambda 등의 AWS 서비스가 이 계정에서 작업을 수행하도록 허용합니다.

☐ AWS 계정

사용자 또는 서드 파티에 속한 다른 AWS 계정의 엔터티가 이 계정에서 작업을 수행하도록 허용합니다.

☐ 웹 자격 증명

지정된 외부 웹 ID 제공업체와 연동된 사용자가 이 역할을 맡아 이 계정에서 작업을 수행하도록 허용합니다.

☐ SAML 2.0 페더레이션

기업 디렉터리에서 SAML 2.0과 연동된 사용자가 이 계정에서 작업을 수행할 수 있도록 허용합니다.

☐ 사용자 지정 신뢰 정책

다른 사용자가 이 계정에서 작업을 수행할 수 있도록 사용자 지정 신뢰 정책을 생성합니다.

사용 사례

EC2, Lambda 등의 AWS 서비스가 이 계정에서 작업을 수행하도록 허용합니다.

서비스 또는 사용 사례

EKS

지정된 서비스에 대한 사용 사례를 선택합니다.

사용 사례

☐ EKS - Service

Allows EKS to manage clusters on your behalf.

☒ EKS - Cluster

Allows the cluster Kubernetes control plane to manage AWS resources on your behalf.

15.4 EKS 생성하기

권한 추가 정보

권한 정책 (1) 정보

선택한 역할 유형에는 다음 정책이 필요합니다.

정책 이름 ↗



유형



[AmazonEKSClusterPolicy](#)

AWS 관리형

▶ 권한 경계 설정 - 선택 사항

취소

이전

다음

15.4 EKS 생성하기

이름 지정, 검토 및 생성

역할 세부 정보

역할 이름

이 역할을 식별하는 의미 있는 이름을 입력합니다.

my-cluster-role

my-cluster-role

최대 64자입니다. 영문자, 숫자, -, ., @, _ 문자를 사용하세요.

설명

이 역할에 대하여 간단한 설명을 추가합니다.

Allows the cluster Kubernetes control plane to manage AWS resources on your behalf.

최대 문자 수: 1000. 문자(A~Z 및 a~z), 숫자(0~9), 탭, 새 줄 또는 다음 문자 중 하나를 사용합니다: _+=, . @-/[\{\}!#\$%^*()~:"'`

3단계: 태그 추가

태그 추가 - 선택 사항 정보

태그는 리소스를 식별, 정리 또는 검색하는 데 도움이 되도록 AWS 리소스에 추가할 수 있는 키-값 페어입니다.

리소스와 연결된 태그가 없습니다.

새 태그 추가

최대 50개의 태그를 더 추가할 수 있습니다.

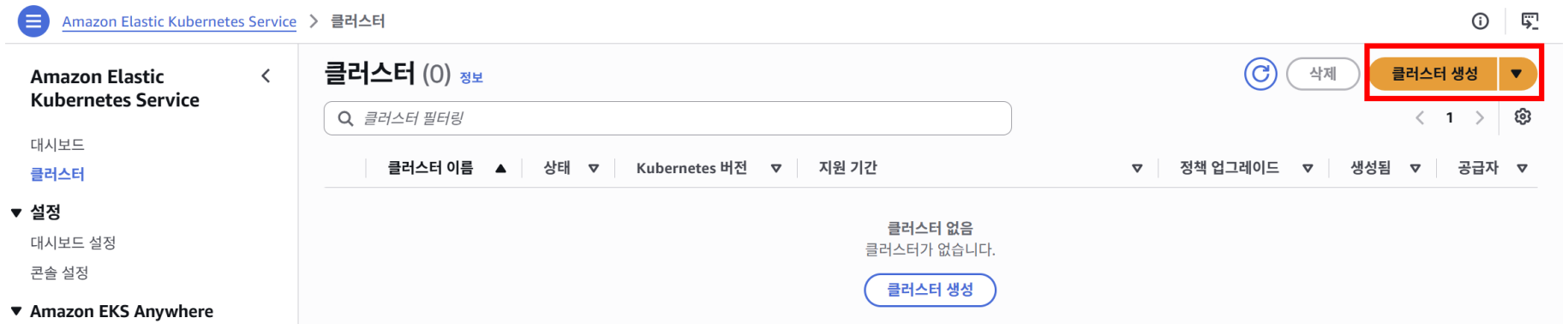
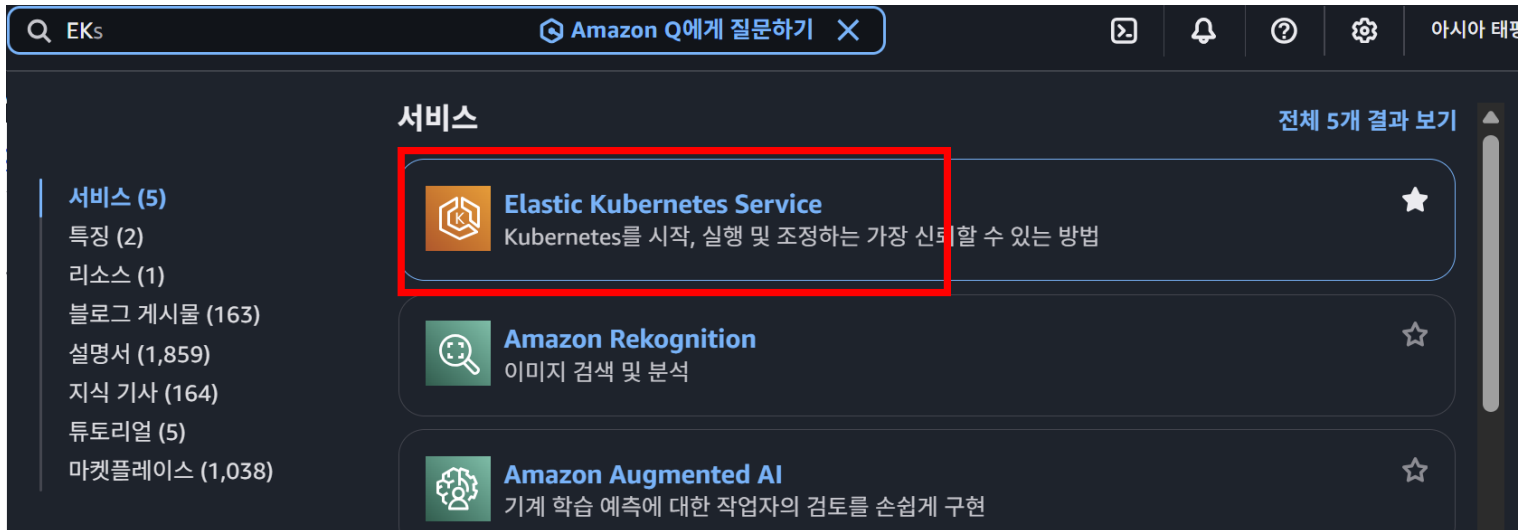
취소

이전

역할 생성

15.4 EKS 생성하기

3. EKS 클러스터 생성



15.4 EKS 생성하기

클러스터 구성

구성 옵션 정보

클러스터를 구성할 방법을 선택합니다.

- ☐ 빠른 구성(EKS 자율 모드 사용)
프로덕션 수준의 기본 설정으로 클러스터를 빠르게 생성할 수 있습니다. 이 구성에서는 EKS 자율 모드를 사용하여 노드 생성 및 스토리지 프로비저닝과 같은 인프라 작업을 자동화합니다.

- ☒ 사용자 지정 구성
생성하기 전에 기본 설정을 변경하려면 이 옵션을 선택합니다. 이 구성은 EKS 자율 모드를 사용하고 클러스터 구성을 사용자 지정할 수 있는 옵션을 제공합니다.

EKS 자율 모드 정보

EKS의 자율 모드를 사용할지 여부를 선택합니다.

- ☒ EKS 자율 모드 사용
EKS는 컴퓨팅, 스토리지, 네트워킹을 위한 일상적인 클러스터 작업을 자동화합니다. 새 포드가 기존 노드에 맞지 않을 경우 EKS는 새 노드를 생성합니다. EKS는 AWS에서 관리하는 클러스터 인프라에 통합 Kubernetes 기능을 결합하여 애플리케이션 컴퓨팅 요구 사항을 충족합니다.
[요금 보기](#)

▶ 포함된 기능

클러스터 구성 정보

이름

이 클러스터의 고유 이름을 입력합니다. 클러스터가 생성된 후에는 이 속성을 변경할 수 없습니다.

my-cluster

my-cluster

클러스터 이름은 문자 또는 숫자로 시작해야 하며 유니코드 문자 세트, 숫자, 하이픈 및 밑줄을 사용할 수 있습니다. 최대 길이는 100자입니다.

15.4 EKS 생성하기

클러스터 구성 정보

이름

이 클러스터의 고유 이름을 입력합니다. 클러스터가 생성된 후에는 이 속성을 변경할 수 없습니다.

my-cluster

클러스터 이름은 문자 또는 숫자로 시작해야 하며 유니코드 문자 세트, 숫자, 하이픈 및 밑줄을 사용할 수 있습니다. 최대 길이는 100자입니다.

클러스터 IAM 역할

다른 AWS 서비스 및 리소스에 액세스하려면 Kubernetes 컨트롤 플레인 IAM 역할이 필요합니다. Kubernetes 컨트롤 플레인에 적합한 권한이 있는 역할을 선택하세요.

my-cluster-role



새 역할 생성

[역할 편집](#)

[IAM에서 역할 세부 정보 보기](#)

나머지 옵션은 그대로 두고 다음 버튼 누르기

태그 (0) 정보

리소스와 연결된 태그가 없습니다.

새 태그 추가

최대 50개의 태그를 더 추가할 수 있습니다.

[취소](#)

다음

15.4 EKS 생성하기

네트워킹 지정

네트워킹 정보

클러스터 생성 후에는 IP 주소 패밀리 및 서비스 IP 주소 범위를 변경할 수 없습니다.

VPC 정보

EKS 클러스터 리소스에 사용할 VPC를 선택합니다.

vpc-078cec7a08d644933 | demo-vpc-inky4832 ▼



기본 설정

서브넷 정보

클러스터와의 원활한 통신을 위해 제어 플레인 이 탄력적 네트워크 인터페이스(ENI)를 배치할 수 있는 서브넷을 VPC 내에서 선택합니다. 새 서브넷을 생성하려면 [VPC 콘솔](#)의 해당 페이지로 이동합니다.

서브넷 선택 ▼



선택한 서브넷 지우기

subnet-0e3a746dc23c879cb | demo-vpc-inky4832-public-subnet-a ✕
ap-northeast-2a 10.25.1.0/24

subnet-0a2e13f6355c04379 | demo-vpc-inky4832-public-subnet-b ✕
ap-northeast-2b 10.25.2.0/24

추가 보안 그룹 정보

EKS는 클러스터 생성 시 클러스터 보안 그룹을 자동으로 생성하여 워커 노드와 컨트롤 플레인 간의 통신을 지원합니다. 필요에 따라 컨트롤 플레인 서브넷에 생성된 EKS 관리형 탄력적 네트워크 인터페이스에 적용할 추가 보안 그룹을 선택할 수도 있습니다. 새 보안 그룹을 생성하려면 [VPC 콘솔](#)의 해당 페이지로 이동합니다.

보안 그룹 선택 ▼



15.4 EKS 생성하기

클러스터 IP 주소 패밀리 선택 [정보](#)

클러스터의 Pod 및 서비스에 대한 IP 주소 유형을 지정합니다.

- ☒ IPv4
☐ IPv6

Kubernetes 서비스 IP 주소 블록 구성 [정보](#)

☒ 클러스터 서비스가 IP 주소를 수신할 범위를 지정합니다.

하이브리드 노드를 활성화하도록 원격 네트워크 구성 [정보](#)

EKS 하이브리드 노드를 사용하면 온프레미스 및 엣지 인프라를 EKS 클러스터의 노드로 사용할 수 있습니다.

☒ 하이브리드 노드에 사용할 온프레미스 환경의 CIDR 블록을 지정합니다.

클러스터 엔드포인트 액세스 [정보](#)

Kubernetes API 서버 엔드포인트에 대한 액세스 권한을 구성합니다.

- ☐ 퍼블릭
 VPC 외부에서 클러스터 엔드포인트에 액세스할 수 있습니다. 작업자 노드 트래픽은 엔드포인트에 연결하기 위해 VPC를 벗어납니다.
- ☒ 퍼블릭 및 프라이빗
 VPC 외부에서 클러스터 엔드포인트에 액세스할 수 있습니다. 엔드포인트에 대한 작업자 노드 트래픽은 VPC 내에 유지됩니다.
- ☐ 프라이빗
 클러스터 엔드포인트는 VPC를 통해서만 액세스할 수 있습니다. 엔드포인트에 대한 작업자 노드 트래픽은 VPC 내에 유지됩니다.

📄 그 외 섹션

컨트롤 플레인 송신 [정보](#)

Kubernetes 컨트롤 플레인 트래픽이 라우팅되는 방법을 선택하세요. 클러스터가 생성된 후에는 AWS 관리형에서 고객 라우팅으로 마이그레이션할 수 있지만 그 반대는 불가능합니다.

송신 모드 [정보](#)

- ☒ AWS 관리형
 컨트롤 플레인 트래픽은 기본 퍼블릭 네트워킹 경로를 따릅니다.
- ☐ 고객 라우팅
 컨트롤 플레인 트래픽은 VPC를 통해 라우팅됩니다.

[취소](#)

[이전](#)

[다음](#)

15.4 EKS 생성하기

▶ 관찰성 정보

지표

Prometheus | [정보](#)

- ☐ Prometheus 지표를 Amazon Managed Service for Prometheus로 전송
Amazon Managed Service for Prometheus를 사용하여 애플리케이션 및 인프라 지표를 모니터링하세요. 이러한 지표에는 시스템 상태 및 성능 데이터가 포함됩니다.

CloudWatch | [정보](#)

- ☐ 애플리케이션 및 인프라 텔레메트리를 Amazon CloudWatch로 전송
Amazon CloudWatch 관찰성 추가 기능을 설치하여 CloudWatch APM에서 애플리케이션 지표를 전송하고 CloudWatch Container Insights에서 인프라 텔레메트리를 전송합니다.

컨테이너 네트워크 관찰성 [정보](#)

네트워크 모니터링 | [정보](#)

- ☐ 네트워크 모니터링 활성화
네트워크 모니터링을 통해 서비스 맵에서 트래픽을 시각화하고, 세부적인 트래픽 분석을 위한 상세 플로우 테이블을 탐색하며, 실시간으로 네트워크 성능 지표를 모니터링할 수 있어 문제 해결이 용이합니다. [더 알아보기](#) 

컨트롤 플레인 로그 [정보](#)

Amazon EKS 제어 플레인에서 CloudWatch Logs로 감사 및 진단 로그를 전송합니다.

- ☐ API 서버
클러스터에 대한 API 요청과 관련된 로그입니다.
- ☐ 감사
Kubernetes API를 통한 클러스터 액세스와 관련된 로그입니다.
- ☐ Authenticator
클러스터에 대한 인증 요청과 관련된 로그입니다.
- ☐ 컨트롤러 관리자
클러스터 컨트롤러 상태와 관련된 로그입니다.
- ☐ 스케줄러
예약 결정과 관련된 로그입니다.

[취소](#)
[이전](#)
[다음](#)

15.4 EKS 생성하기

추가 가능 선택

여러 범주의 추가 기능을 검토한 다음 추가 기능을 선택하여 클러스터를 개선합니다.

기본 설정

AWS 추가 기능 (21) 정보



kube-proxy 정보

클러스터 내에서 포트 네트워킹을 활성화합니다.

카테고리
networking

호환되는 컴퓨팅
EC2, 하이브리드 노드 및 로컬 클러스터



CoreDNS 정보

클러스터 내에서 서비스 검색을 활성화합니다.

카테고리
networking

호환되는 컴퓨팅
EC2, 하이브리드 노드, Fargate, EKS 자율 모드 및 로컬 클러스터



Amazon VPC CNI 정보

클러스터 내에서 포트 네트워킹을 활성화합니다.

카테고리
networking

호환되는 컴퓨팅
EC2 및 로컬 클러스터



노드 모니터링 에이전트 정보

노드 상태 문제의 자동 탐지를 활성화합니다.

카테고리
observability

호환되는 컴퓨팅
EC2 및 하이브리드 노드



Amazon EC2 로컬 인스턴스 저장소 CSI 드라이버

클러스터 내에서 Amazon EC2 인스턴스 저장소를 관리합니다.

카테고리
storage

호환되는 컴퓨팅
EC2



CSI 스냅샷 컨트롤러 정보

클러스터 내 CSI 드라이버에서 스냅샷 기능을 활성화합니다.

카테고리
storage

호환되는 컴퓨팅
EC2, 하이브리드 노드, EKS 자율 모드 및 Fargate



AWS Private CA Connector for Kubernetes 정보

AWS Private CA에서 X.509 인증서를 발행하도록 cert-manager를 활성화합니다. cert-manager가 필요합니다.

카테고리
security



Amazon EBS CSI 드라이버 정보

클러스터 내에서 Amazon Elastic Block Storage(EBS)를 활성화합니다.

카테고리
storage

호환되는 컴퓨팅



AWS Secrets Store CSI 드라이버 제공업체

AWS Secrets Manager에서 암호를 검색하고 AWS Systems Manager Parameter Store에서 파라미터를 검색하고 Kubernetes 포트에 파일로 마운트합니다.

카테고리
security

15.4 EKS 생성하기

AWS Marketplace 추가 기능 (0) [정보](#)

EKS 클러스터를 개선하기 위해 EKS 추가 기능을 검색, 구독 및 구성합니다.

필터링 옵션

모든 범주 ▼

모든 공급업체 ▼

모든 요금 모델 ▼

1.36 ▼

필터 지우기

< 1 >

일치 없음
검색과 일치하는 항목이 없습니다.

취소

이전

다음

선택한 추가 기능 설정 구성

설정을 선택하여 클러스터에 대한 추가 기능을 구성합니다.

기본 설정

kube-proxy 정보

추가 기능 제거

등록자



카테고리

networking

상태

✓ 설치 준비 완료

버전

이 추가 기능의 버전을 선택합니다.

v1.36.0-eksbuild.7

▶ 선택적 구성 설정

지표 서버

추가 기능 제거

등록자



카테고리

observability

상태

✓ 설치 준비 완료

버전

이 추가 기능의 버전을 선택합니다.

v0.8.1-eksbuild.11

▶ 선택적 구성 설정

취소

이전

다음

15.4 EKS 생성하기

검토 및 생성

1단계: 클러스터

기본 설정

편집

클러스터 구성

이름

kube-practice

EKS 자율 모드

비활성화됨

클러스터 IAM 역할

arn:aws:iam::727646470302:role/kube-cluster-role

인증 모드

EKS API

Kubernetes 버전

1.36

정책 업그레이드

표준 지원

Kubernetes 클러스터 관리자 액세스

클러스터 관리자 액세스 허용

컨트롤 플레인 스케일링 티어

표준

EKS Pod Identity (2)

< 1 >

추가 기능 이름



IAM 역할 [↗](#)



서비스 계정



external-dns

설정되지 않음

external-dns

vpc-cni

설정되지 않음

aws-node

취소

이전

생성

15.4 EKS 생성하기

Amazon Elastic Kubernetes Service

대시보드

클러스터

▼ 설정

대시보드 설정

콘솔 설정

▼ Amazon EKS Anywhere

Enterprise 구독

▼ 관련 서비스

Amazon ECR

AWS Batch

설명서

EKS 워크숍

kube-practice



연결

작업

모니터 클러스터

Amazon EKS 콘솔은 클러스터가 ACTIVE 또는 UPDATING 상태가 될 때까지 Kubernetes 리소스를 표시하지 않습니다. 몇 분 후에 다시 시도하세요.

▼ 클러스터 정보

상태

🔄 생성 중

Kubernetes 버전
1.36지원 기간
2027년 8월 2일까지 표준 지원공급자
EKS

클러스터 상태

🟢 0

업그레이드 인사이트

🟢 0

노드 상태 문제

🟢 0

기능 문제

🟢 0

개요

리소스

컴퓨팅

네트워킹

추가 기능

기능

액세스

관찰성

업데이트 기록 및 백업

태그

세부 정보

API 서버 엔드포인트

OpenID Connect 공급자 URL

생성됨

🕒 몇 초 전



Amazon Elastic Kubernetes Service

대시보드

클러스터

▼ 설정

대시보드 설정

콘솔 설정

▼ Amazon EKS Anywhere

Enterprise 구독

▼ 관련 서비스

Amazon ECR

AWS Batch

설명서

EKS 워크숍

kube-practice



연결

작업

모니터 클러스터

▼ 클러스터 정보

상태

🟢 활성

Kubernetes 버전
1.36지원 기간
2027년 8월 2일까지 표준 지원공급자
EKS

클러스터 상태

🟢 0

업그레이드 인사이트

🟢 4

노드 상태 문제

🟢 0

기능 문제

🟢 0

개요

리소스

컴퓨팅

네트워킹

추가 기능 1

기능

액세스

관찰성

업데이트 기록 및 백업

태그

세부 정보

API 서버 엔드포인트

🔗 <https://3D9BBC2CAE78BC7C1F48670D7E366488.gr7.ap-northeast-2.eks.amazonaws.com>

OpenID Connect 공급자 URL

🔗 <https://oidc.eks.ap-northeast-2.amazonaws.com/id/3D9BBC2CAE78BC7C1F48670D7E366488>

생성됨

🕒 2시간 전

인증 기관

🔗 LS0tLS1CRUdJTiBDRVJUSUJZQ0FURS0tLS0tCk1JSURCVENDQWUyZ0F3SUJBZ0lUSDUxa215OVRFSmt3RFFZSkVWklodmNOQVVF

클러스터 IAM 역할 ARN

🔗 <arn:aws:iam::727646470302:role/kube-cluster-role>

클러스터 ARN

🔗 <arn:aws:eks:ap-northeast-2:727646470302:cluster/kube-practice>

플랫폼 버전 정보

eks.7

15.4 EKS 생성하기

4. EKS 워커 노드 그룹 추가하기

kube-practice

🔄

🔗 연결

⌵ 작업

모니터 클러스터

▼ 클러스터 정보 정보

상태

🟢 활성

클러스터 상태

🟢 0

Kubernetes 버전

1.36

업그레이드 인사이트

🟢 4

지원 기간

🕒 2027년 8월 2일까지 표준 지원

노드 상태 문제

🟢 0

공급자

EKS

기능 문제

🟢 0

개요

리소스

컴퓨팅

네트워킹

추가 기능 1

기능

액세스

관찰성

업데이트 기록 및 백업

태그

노드 (0) 정보

🔍 속성 또는 값으로 노드 필터링

< 1 >

노드 이름

▲

인스턴스 유형

▼

컴퓨팅

▼

관리 주체

▼

생성됨

▼

상태

▼

CPU 사용량

메모리 사용량

임시 스토리지 사용량

노드 없음

이 클러스터에 노드(가) 없거나 해당 클러스터를 볼 수 있는 권한이 없습니다.

노드 그룹 (0) 정보

노드 그룹은 EC2 Auto Scaling 그룹을 통해 기본 컴퓨팅 스케일링을 구현합니다.

🔍 속성 또는 값을 기준으로 노드 그룹 필터링

< 1 >

그룹 이름

▲

원하는 크기

▼

AMI 릴리스 버전

▼

시작 템플릿

▼

상태

▼

노드 그룹 없음

이 클러스터에는 노드 그룹이 없습니다.

Amazon EKS 관리형 노드 그룹이 아닌 노드 AWS 콘솔에 표시되지 않습니다.

노드 그룹 추가

41

15.4 EKS 생성하기

노드 그룹 구성 정보

노드 그룹은 Amazon EKS 클러스터에 컴퓨팅 용량을 제공하는 EC2 인스턴스의 그룹입니다. 클러스터에는 여러 노드 그룹을 추가할 수 있습니다.

노드 그룹 구성

노드 그룹을 생성한 후에는 이러한 속성을 변경할 수 없습니다.

이름

이 노드 그룹에 대한 고유한 이름을 설정합니다.

my-nodes

my-nodes

노드 그룹 이름은 문자 또는 숫자로 시작해야 하며 유니코드 문자 세트, 숫자, 하이픈 및 밑줄을 포함할 수 있습니다. 최대 길이는 63자입니다.

노드 IAM 역할

다른 AWS 서비스 및 리소스에 액세스하려면 노드에 IAM 역할이 필요합니다. 노드에 적합한 권한이 있는 역할을 선택하세요.

기존 역할 선택



새 역할 생성

15.4 EKS 생성하기

역할 생성

역할 세부 정보

역할 이름

my-node-group-role

my-node-group-role

최대 64자입니다. 영숫자를 사용하여 +, -, @, _ 자로 생성하세요.

사용 사례

이 역할을 통해 EKS 노드(가) 사용자를 대신하여 AWS 서비스를 호출할 수 있습니다. [신뢰 정책 보기](#).

기본 정책 (4)

이러한 정책은 역할에 연결되며 EKS 노드에서 다른 서비스, API 및 계정에 액세스하거나 통신할 수 있는 권한을 정의합니다.

정책 이름 ↗



유형

	AmazonEC2ContainerRegistryReadOnly	AWS 관리형
	AmazonEKS_CNI_Policy	AWS 관리형
	AmazonEKSWorkerNodePolicy	AWS 관리형
	AmazonElasticContainerRegistryPublicReadOnly	AWS 관리형

이미 정책이 추가됨



설명

작업

Provides read-only access to Amazon E...	-
This policy provides the Amazon VPC ...	-
This policy allows Amazon EKS worker ...	-
Provides read-only access to Amazon E...	제거

▶ 고급

[취소](#)
[역할 생성](#)

15.4 EKS 생성하기

노드 그룹 구성

정보

노드 그룹은 Amazon EKS 클러스터에 컴퓨팅 용량을 제공하는 EC2 인스턴스의 그룹입니다. 클러스터에는 여러 노드 그룹을 추가할 수 있습니다.

노드 그룹 구성

노드 그룹을 생성한 후에는 이러한 속성을 변경할 수 없습니다.

이름

이름을 할당합니다.

my-nodes

숫자로 시작해야 하며 유니코드 문자, 계층, 공백, 밑줄, 하이픈 및 점만을 포함할 수 있습니다. 최대 길이는 63자입니다.

노드 IAM 역할

다른 AWS 서비스 및 리소스에 액세스하려면 노드에 IAM 역할이 필요합니다. 노드에 적합한 권한이 있는 역할을 선택하세요.

새 역할 my-node-group-role 성공

my-node-group-role

새 역할 생성

역할 편집

IAM에서 역할 세부 정보 보기

시작 템플릿

정보

노드 그룹을 생성한 후에는 이러한 속성을 변경할 수 없습니다.

☒ 시작 템플릿 사용

EC2 시작 템플릿을 사용하여 이 노드 그룹을 구성합니다.

Kubernetes 레이블

정보

이 노드 그룹에는 레이블이 없습니다.

레이블 추가

추가할 수 있는 나머지 레이블: 50

Kubernetes 테인트

정보

이 노드 그룹에는 테인트가 없습니다.

테인트 추가

추가할 수 있는 나머지 테인트: 50

태그

정보

리소스와 연결된 태그가 없습니다.

새 태그 추가

최대 50개의 태그를 더 추가할 수 있습니다.

취소

다음

15.4 EKS 생성하기

컴퓨팅 및 조정 구성 설정

노드 그룹 컴퓨팅 구성 **Amazon Linux 2023(x86_64) Standard(AL2023_x86_64)**

노드 그룹을 생성한 후에는 이러한 속성을 변경할 수 없습니다.

AMI 유형 [정보](#)

노드에 대한 EKS 최적화 Amazon Machine Image를 선택합니다.

Amazon Linux 2023(x86_64) Standard (AL2023_x86_64_STANDARD) ▼

용량 유형

이 노드 그룹에 대한 용량 구매 옵션을 선택합니다.

On-Demand ▼

인스턴스 유형 [정보](#)

이 노드 그룹에 대해 선호하는 인스턴스 유형을 선택합니다.

🔍 인스턴스 유형 입력

t3.large

vCPU: 2 vCPUs Memory: 8 GiB Network: Up to 5 Gigabit Max ENI: 3 Max IPs: 36

t3.large

디스크 크기

각 노드에 연결되는 EBS 볼륨의 크기를 선택합니다.

20

GiB

15.4 EKS 생성하기

노드 그룹 조정 구성

원하는 크기
그룹에서 처음에 시작할 노드 수를 설정합니다.

2 노드

원하는 노드 크기는 0보다 크거나 같아야 함

최소 크기
그룹에서 축소할 수 있는 최소 노드 수를 설정합니다.

2 노드

최소 노드 크기는 0보다 크거나 같아야 함

최대 크기
그룹에서 확장할 수 있는 최대 노드 수를 설정합니다.

2 노드

최대 노드 크기는 1보다 크거나 같아야 하며 최소 크기보다 작을 수 없음

AWS Auto Scaling(ASG) 원 플

☒ ASG 원 플 사용
스케일 아웃 이벤트 중에 노드 프로비저닝 시간을 단축합니다.

노드 그룹 업데이트 구성 정보

최대 사용 불가
노드 그룹 버전 업데이트 중에 사용할 수 없는 노드의 최대 허용 수 또는 백분율을 설정합니다.

☒ 수
숫자 입력

☐ 백분율
백분율 지정

값
1 노드
노드 수는 0보다 커야 합니다.

전략 업데이트
☒ 기본값
☐ 최소

노드 자동 복구 구성 정보

노드 자동 복구가 활성화되면 Amazon EKS는 관리형 노드 그룹 내 노드의 상태를 지속적으로 모니터링합니다. 이 기능은 문제가 발생한 노드를 자동으로 감지하여 교체합니다.

☒ 노드 자동 복구 기능은 Kubelet의 준비 상태와 모든 노드 객체 수동 삭제에 반응합니다. 노드 모니터링 에이전트도 함께 설치하면 더 많은 복구 대상 노드 조건을 감지할 수 있습니다. [추가 기능으로 이동](#)

☐ 노드 자동 복구 활성화

취소

이전

다음

15.4 EKS 생성하기

네트워킹 지정

노드 그룹 네트워크 구성

노드 그룹을 생성한 후에는 이러한 속성을 변경할 수 없습니다.

서브넷 정보

노드가 실행될 VPC의 서브넷을 지정합니다. 새 서브넷을 생성하려면 [VPC 콘솔](#)의 해당 페이지로 이동합니다.

서브넷 선택



선택한 서브넷 지우기

subnet-0e3a746dc23c879cb | demo-vpc-inky4832-public-subnet-a ✕
ap-northeast-2a 10.25.1.0/24

subnet-0a2e13f6355c04379 | demo-vpc-inky4832-public-subnet-b ✕
ap-northeast-2b 10.25.2.0/24

☐ 노드에 대한 원격 액세스 허용 [정보](#)

취소

이전

다음

검토 및 생성

1단계: 노드 그룹

기본 설정

편집

노드 그룹 구성

이름
kube-practice-nodes

노드 IAM 역할
arn:aws:iam::727646470302:role/kube-node-group-role

Kubernetes 레이블 (0)

< 1 >

키 ▼ | 값 ▼

레이블 없음
이 노드 그룹에는 Kubernetes 레이블이 없습니다.

3단계: 네트워킹

편집

노드 그룹 네트워크 구성

서브넷
subnet-0e3a746dc23c879cb
subnet-0a2e13f6355c04379

노드에 대한 원격 액세스 허용
off

취소

이전

생성

5. 노드 그룹 생성 확인

my-nodes

노드 그룹 구성 정보

Kubernetes 버전
1.36

AMI 릴리스 버전 정보
1.36.2-20260709

AMI 유형 정보
Amazon Linux 2023(x86_64) Standard

인스턴스 유형
t3.large

상태
⋯ 생성 중

디스크 크기
20 GiB

로컬 버전

편집

삭제

my-nodes

노드 그룹 구성 정보

Kubernetes 버전
1.36

AMI 릴리스 버전 정보
1.36.2-20260709

AMI 유형 정보
Amazon Linux 2023(x86_64) Standard

인스턴스 유형
t3.large

상태
✔️ 활성

디스크 크기
20 GiB

로컬 버전

편집

삭제

49

15.4 EKS 생성하기

my-nodes



롤백 버전

편집

삭제

노드 그룹 구성 정보

Kubernetes 버전
1.36

AMI 유형 정보
Amazon Linux 2023(x86_64) Standard

상태
🟢 활성

AMI 릴리스 버전 정보
1.36.2-20260709

인스턴스 유형
t3.large

디스크 크기
20 GiB

세부 정보

노드

상태 문제 0

Kubernetes 레이블

업데이트 구성

Kubernetes 테인트

업데이트 기록

태그

노드 (2) 정보

 Q 속성 또는 값으로 노드 필터링

< 1 >

노드 이름	인스턴스 유형	컴퓨팅	관리 주체	생성됨	상태	CPU 사용량	메모리 사용량
ip-10-25-1-86.ap-northeast-2.compute.internal	t3.large	Node group	my-nodes	1분 전	🟢 준비 완료	32%	17%
ip-10-25-2-191.ap-northeast-2.compute.internal	t3.large	Node group	my-nodes	1분 전	🟢 준비 완료	12%	9%

15.4 EKS 생성하기

6. 생성된 EC2 인스턴스 확인하기

-하나의 워커 노드(worker node)가 하나의 EC2 인스턴스에서 실행됨

인스턴스 (2) 정보

최종 업데이트 날짜
less than a minute 전

연결

인스턴스 상태 ▼

작업 ▼

인스턴스 시작 ▼

저장한 필터 세트
필터 세트 선택 ▼

Q

인스턴스를 속성 또는 (case-sensitive) 태그로 찾기

<

1

>

⚙

☐	Name	인스턴스 ID	인스턴스 상태	인스턴스 유형	상태 검사	가용 영역	퍼블릭 IPv4 DNS	
☐		i-0184cc6d090094270	✓ 실행 중	t3.large	🕒 초기화	ap-northeast-2a	ec2-52-78-118-241.ap-...	5
☐		i-08e000e9a54b270f8	✓ 실행 중	t3.large	✓ 3/3 checks passed	ap-northeast-2b	ec2-43-203-191-119.ap...	4

15.4 EKS 생성하기

7. 현재 kubectl이 연동되는 클러스터 정보 보기

```
kubectl config get-contexts
```

```
C:\Users\User>kubectl config get-contexts
CURRENT   NAME             CLUSTER           AUTHINFO           NAMESPACE
*         docker-desktop   docker-desktop    docker-desktop
C:\Users\User>
```

15.4 EKS 생성하기

8. kubectl 에 EKS 클러스터 추가하기

```
# aws eks --region ap-northeast-2 update-kubeconfig --name EKS 클러스터  
이름 >
```

```
aws eks --region ap-northeast-2 update-kubeconfig --name my-cluster
```

```
C:\Users\User>aws eks --region ap-northeast-2 update-kubeconfig --name my-cluster  
Added new context arn:aws:eks:ap-northeast-2:727646470302:cluster/my-cluster to C:\Users\User\.kube\config  
  
C:\Users\User>kubectl config get-contexts  
CURRENT  NAME                                     CLUSTER                                     NAMESPACE  
*         arn:aws:eks:ap-northeast-2:727646470302:cluster/my-cluster  arn:aws:eks:ap-northeast-2:727646470302:cluster/m  
y-cluster  arn:aws:eks:ap-northeast-2:727646470302:cluster/my-cluster  
docker-desktop  
docker-desktop  
docker-desktop  
  
C:\Users\User>
```

15.4 EKS 생성하기

9. 제대로 접속했는지 확인하기

```
C:\Users\User>kubectl get nodes
NAME                                                    STATUS    ROLES    AGE   VERSION
ip-10-25-1-86.ap-northeast-2.compute.internal        Ready     <none>    20m   v1.36.2-eks-8f14419
ip-10-25-2-191.ap-northeast-2.compute.internal        Ready     <none>    20m   v1.36.2-eks-8f14419
C:\Users\User>
```

15.4 EKS 생성하기

10. 로컬(윈도우)에 있는 deploy_AWS.yaml 수정하기

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: account-backend

spec:
  revisionHistoryLimit: 2
  replicas: 1

selector:
  matchLabels:
    app: account-backend
```


15.4 EKS 생성하기

```
template:
  metadata:
    labels:
      app: account-backend

  spec:
    containers:
      - name: account-backend
        image: 727646470302.dkr.ecr.ap-northeast-2.amazonaws.com/my-ecr:1.0 # ECR에 업로드 된 URI 값으로 수정
        imagePullPolicy: IfNotPresent

    ports:
      - containerPort: 8080

    resources:
      limits:
        cpu: "1"
        memory: "500Mi"
      requests:
        cpu: "0.5"
        memory: "250Mi"

    env:
      - name: DB_HOST
        valueFrom:
          secretKeyRef:
            name: my-app-secrets
            key: DB_HOST
```

15.4 EKS 생성하기

```
- name: DB_PORT
  valueFrom:
    secretKeyRef:
      name: my-app-secrets
      key: DB_PORT

- name: DB_NAME
  valueFrom:
    secretKeyRef:
      name: my-app-secrets
      key: DB_NAME

- name: DB_USERNAME
  valueFrom:
    secretKeyRef:
      name: my-app-secrets
      key: DB_USERNAME

- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: my-app-secrets
      key: DB_PASSWORD
```

```
livenessProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 8080
    initialDelaySeconds: 100
    periodSeconds: 20

readinessProbe:
  httpGet:
    path: /actuator/health/readiness
    port: 8080
    initialDelaySeconds: 100
    periodSeconds: 10
```

15.4 EKS 생성하기

```
---  
apiVersion: v1  
kind: Service  
metadata:  
  name: account-backend-service  
  
spec:  
  type: ClusterIP  
  
  ports:  
    - port: 80  
      targetPort: 8080  
  
  selector:  
    app: account-backend
```

15.4 EKS 생성하기

11. Ingress Controller 설치

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.8.1/deploy/static/provider/aws/deploy.yaml
```

```
C:\Users\User>kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.8.1/deploy/static/provider/aws/deploy.yaml
namespace/ingress-nginx created
serviceaccount/ingress-nginx created
serviceaccount/ingress-nginx-admission created
role.rbac.authorization.k8s.io/ingress-nginx created
role.rbac.authorization.k8s.io/ingress-nginx-admission created
clusterrole.rbac.authorization.k8s.io/ingress-nginx created
clusterrole.rbac.authorization.k8s.io/ingress-nginx-admission created
rolebinding.rbac.authorization.k8s.io/ingress-nginx created
rolebinding.rbac.authorization.k8s.io/ingress-nginx-admission created
clusterrolebinding.rbac.authorization.k8s.io/ingress-nginx created
clusterrolebinding.rbac.authorization.k8s.io/ingress-nginx-admission created
configmap/ingress-nginx-controller created
service/ingress-nginx-controller created
service/ingress-nginx-controller-admission created
deployment.apps/ingress-nginx-controller created
job.batch/ingress-nginx-admission-create created
job.batch/ingress-nginx-admission-patch created
ingressclass.networking.k8s.io/nginx created
validatingwebhookconfiguration.admissionregistration.k8s.io/ingress-nginx-admission created
```

15.4 EKS 생성하기

로드 밸런서 새로운 기능

Elastic Load Balancing은 수신 트래픽의 변화에 따라 자동으로 로드 밸런서 용량을 확장합니다.

로드 밸런서 필터링

이름 상태 유형 체계 IP 주소 유형 VPC ID ↗




로드 밸런서 (1) 새로운 기능

Elastic Load Balancing은 수신 트래픽의 변화에 따라 자동으로 로드 밸런서 용량을 확장합니다.

로드 밸런서 필터링

이름 상태 유형 체계 IP 주소 유형 VPC ID ↗

☐	a30884b7ca9ef483581...	프로비저닝 중	network	Internet-facing	IPv4	vpc-078cec7a08d64493...
--------------------------	------------------------	---------	---------	-----------------	------	-------------------------

ALB 아님

15.4 EKS 생성하기

12. NLB를 Route 53에 매핑

Route53에 가서 ingress Controller에 의해서 자동으로 생성된 LB의 도메인을 inky4832.click 도메인과 매핑

a9fbd162c38e8495fb5323a80aa294a8
🔄
작업 ▼

▼ 세부 정보

로드 밸런서 유형 네트워크	상태 ✅ 활성	VPC vpc-078cec7a08d644933	로드 밸런서 IP 주소 유형 IPv4
체계 Internet-facing	호스팅 영역 ZIBE1TIR4HY56	가용 영역 subnet-0e3a746dc23c879cb ap-northeast-2a (apne2-az1) subnet-0a2e13f6355c04379 ap-northeast-2b (apne2-az2)	생성된 날짜 2026년 7월 12일, 19:57 (UTC+09:00)

로드 밸런서 ARN
 arn:aws:elasticloadbalancing:ap-northeast-2:727646470302:loadbalancer/net/a9fbd162c38e8495fb5323a80aa294a8/3e36bdaa16401728

DNS 이름 정보
 a9fbd162c38e8495fb5323a80aa294a8-3e36bdaa16401728.elb.ap-northeast-2.amazonaws.com (A 레코드)

15.4 EKS 생성하기

퍼블릭

inky4832.click

정보

영역 삭제

레코드 테스트

쿼리 로깅 구성

▶ 호스팅 영역 세부 정보

호스팅 영역 편집

레코드(14)

가속 복구

DNSSEC 서명

호스팅 영역 태그(0)

레코드 (14) 정보

레코드 삭제

영역 파일 가져오기

레코드 생성

Automatic 모드는 최상의 필터 결과에 최적화된 현재 검색 동작입니다. [모드를 변경하려면 설정\(settings\)으로 이동합니다.](#)

Q 속성 또는 값을 기준으로 레코드 필터링

유형 ▼

라우팅 정책 ▼

별칭 ▼

< 1 >

⚙

☐	레코드 ... ▼	유형 ▼	라우팅 ... ▼	차별화... ▼	별칭 ▼	값/트래픽 라우팅 대상 ▼	TTL(초) ▼
☐	inky4832....	NS	단순	-	아니요	ns-1963.awsdns-53.co.uk. ns-510.awsdns-63.com. ns-1301.awsdns-34.org. ns-705.awsdns-24.net.	172,800

15.4 EKS 생성하기

레코드 생성

빠른 레코드 생성

▼ 레코드 1

레코드 이름

app

레코드 유형

A - IPv4 주소 및 일부 AWS 리소스로 트래픽 라우팅

별칭

트래픽 라우팅 대상

Network Load Balancer에 대한 별칭

아시아 태평양(서울)

a30884b7ca9ef483581ad83c851e4f38-ed61a1d78f3d626.elb.ap-northeast-2.amazonaws.com

별칭 호스팅 영역 ID: ZIBE1TIR4HY56

라우팅 정책

단순 라우팅

대상 상태 평가

예

마법사로 전환

삭제

다른 레코드 추가

취소

레코드 생성

Network Load Balancer에 대한 별칭 선택

LB의 DNS 값 선택

63

15.4 EKS 생성하기

14. 로컬(윈도우)에 있는 ingress_AWS.yaml 수정하기

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: account-backend-ingress
spec:
  ingressClassName: nginx

  rules:
    - host: app.inky4832.click # Route 53에서 레코드 생성
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: account-backend-service
                port:
                  number: 80
```

15.4 EKS 생성하기

15. 생성

```
kubectl apply -f db_secret_AWS.yaml  
kubectl apply -f ingress_AWS.yaml  
kubectl apply -f redis_AWS.yaml  
kubectl apply -f deploy_AWS.yaml
```

16. 확인

```
kubectl get secret  
kubectl get deployment  
kubectl get pods  
kubectl get services  
  
kubectl get ingress
```

15.4 EKS 생성하기

회원가입및account생성 **http://app.inky4832.click/api/signup-accounts**

METHOD: **POST** SCHEME://HOST[:PORT]/PATH[?QUERY]

http://app.inky4832.click/api/signup-accounts length: 45 byte(s)

Send

QUERY PARAMETERS

HEADERS ⑦

☒ Content-Type: application/json

+ Add header Add authorization

BODY ⑦

```
{
  "username": "인경열",
  "password": "1234",
  "email": "inky@test.com",
  "mobileNumber": "01022229999"
}
```

Response Elapsed Time: 164ms

201

HEADERS ⑦

Date: Sun, 12 Jul 2026 13:27:29 GMT -1
h 1m

Content-Type: application/json

Transfer-Enc... chunked

Connection: keep-alive

Vary: Origin

Vary: Access-Control-Request-Method

Vary: Access-Control-Request-Headers

BODY ⑦

```
{
  statusCode: "201",
  statusMsg: "계정이 성공적으로 생성되었습니다."
}
```

length: 83 bytes

15.4 EKS 생성하기

The screenshot displays a REST client interface with the following details:

- Request:**
 - URL:** `http://app.inky4832.click/auth/login`
 - Method:** `POST`
 - Body:**

```
{
  "email": "inky@test.com",
  "password": "1234"
}
```
- Response:**
 - Status:** `200`
 - Body:**

```
{
  id: 4,
  token: "eyJhbGciOiJIUzUxMiJ9.eyJzdWIiOiJpbmt5QHRlc3QuY29t",
  refreshToken: "eyJhbGciOiJIUzUxMiJ9.eyJzdWIiOiJpbmt5QHRlc3QuY29t"
}
```

Additional visible details include headers for the response (Date, Content-Type, Transfer-Encoding, Connection, Vary) and a 'Save' button for the request.

15.4 EKS 생성하기

account조회 **http://app.inky4832.click/api/accounts?mobileNumber=01011112222** Save

METHOD GET **http://app.inky4832.click/api/accounts?mobileNumber=01011112222** Send length: 63 byte(s)

QUERY PARAMETERS [1]

HEADERS ⑦ 14 Form

Authorization : Bearer eyJhbGciOiJIUzUxMi,

BODY ⑦ XHR does not allow payloads for GET request.

Response Elapsed Time: 106ms

200

HEADERS ⑦ pretty

Date: Sun, 12 Jul 2026 13:30:27 GMT -1h 2m
 Content-Type: application/json
 Transfer-Encoding: chunked
 Connection: keep-alive
 Vary: Origin
 Vary: Access-Control-Request-Method
 Vary: Access-Control-Request-Headers
 X-Content-Type-Options: nosniff
 X-XSS-Protection: 0
 Cache-Control: no-cache, no-store, max-age=0, must-revalidate
 Pragma: no-cache
 Expires: 0

BODY ⑦ pretty

```
{
  customerId: 1,
  username: "유관순3",
  email: "ryu3@test.com",
  password: null,
  role: "USER",
  mobileNumber: "01011112222",
  accountsDto: {
    accountNumber: 1735840403,
    accountType: "저축",
    branchAddress: "서울시 강남구 테헤란로"
  }
}
```

15.5 EKS + https

15.5 EKS + https 적용

1. 로컬에서 Spring Boot 프로젝트를 Docker 이미지로 빌드한 후에 ECR로 Push 하기

```
c:\app-monolithic> aws ecr get-login-password --region ap-northeast-2 | docker  
login --username AWS --password-stdin 727646470302.dkr.ecr.ap-northeast-  
2.amazonaws.com
```

반드시 다음 명령어 실행하기전에 Docker desktop 먼저 실행하자

```
c:\app-monolithic > docker build -t my-ecr .
```

주의: 태그명에 latest 로 하지 말고 구체적인 버전명을 작성한다.

```
c:\app-monolithic > docker tag my-ecr:latest 727646470302.dkr.ecr.ap-  
northeast-2.amazonaws.com/my-ecr:1.0
```

```
c:\app-monolithic > docker push 727646470302.dkr.ecr.ap-northeast-  
2.amazonaws.com/my-ecr:1.0
```

15.5 EKS + https 적용

2. https 통신을 위한 인증서 작업 (Cert-manager 생성)

```
kubectl apply -f https://github.com/cert-manager/cert-manager/releases/download/v1.5.0/cert-manager.crds.yaml
```

```
kubectl apply -f https://github.com/cert-manager/cert-manager/releases/download/v1.5.0/cert-manager.yaml
```

```
PS C:\cloud_engineering\docker\실습소스_제공파일\app_monolithic2_https> kubectl apply -f https://github.com/cert-manager/cert-manager/releases/download/v1.5.0/cert-manager.crds.yaml
customresourcedefinition.apiextensions.k8s.io/certificaterequests.cert-manager.io created
customresourcedefinition.apiextensions.k8s.io/certificates.cert-manager.io created
customresourcedefinition.apiextensions.k8s.io/challenges.acme.cert-manager.io created
customresourcedefinition.apiextensions.k8s.io/clusterissuers.cert-manager.io created
customresourcedefinition.apiextensions.k8s.io/issuers.cert-manager.io created
customresourcedefinition.apiextensions.k8s.io/orders.acme.cert-manager.io created
PS C:\cloud_engineering\docker\실습소스_제공파일\app_monolithic2_https>
PS C:\cloud_engineering\docker\실습소스_제공파일\app_monolithic2_https> kubectl apply -f https://github.com/cert-manager/cert-manager/releases/download/v1.5.0/cert-manager.yaml
customresourcedefinition.apiextensions.k8s.io/certificaterequests.cert-manager.io configured
customresourcedefinition.apiextensions.k8s.io/certificates.cert-manager.io configured
customresourcedefinition.apiextensions.k8s.io/challenges.acme.cert-manager.io configured
customresourcedefinition.apiextensions.k8s.io/clusterissuers.cert-manager.io configured
customresourcedefinition.apiextensions.k8s.io/issuers.cert-manager.io configured
customresourcedefinition.apiextensions.k8s.io/orders.acme.cert-manager.io configured
namespace/cert-manager created
serviceaccount/cert-manager-cainjector created
serviceaccount/cert-manager created
serviceaccount/cert-manager-webhook created
clusterrole.rbac.authorization.k8s.io/cert-manager-cainjector created
```


15.5 EKS + https 적용

3. Redis 생성 (redis_https_AWS.yaml)

```
kubectl apply -f redis_https_AWS.yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis
spec:
  replicas: 1
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
        - name: redis
          image: redis:7-alpine
          ports:
            - containerPort: 6379
```

```
---
apiVersion: v1
kind: Service
metadata:
  name: redis
spec:
  ports:
    - port: 6379
      targetPort: 6379
  selector:
    app: redis
```

15.5 EKS + https 적용

4. Secret 생성 (db_secret_https_AWS.yaml)

```
kubectl apply -f db_secret_https_AWS.yaml
```

```
apiVersion: v1
```

```
kind: Secret
```

```
type: Opaque # 범용
```

```
# Secret 기본 정보
```

```
metadata:
```

```
  name: my-app-secrets # Secret 이름
```

```
# Key, Value 형식으로 값 저장
```

```
stringData:
```

```
  DB_HOST: my-database.cnqmcq6uwqa3.ap-northeast-2.rds.amazonaws.com
```

```
  DB_PORT: "3306"
```

```
  DB_NAME: testdb
```

```
  DB_USERNAME: admin
```

```
  DB_PASSWORD: password
```

15.5 EKS + https 적용

5. Cert 생성 (https_Cert_AWS.yaml)

```
kubectl apply -f https_Cert_AWS.yaml
```

```
apiVersion: cert-manager.io/v1
```

```
kind: ClusterIssuer
```

```
metadata:
```

```
  name: my-issuer
```

```
spec:
```

```
  acme:
```

```
    # ca 서버 주소. 해당 서버를 통해 인증서 발행
```

```
    server: https://acme-v02.api.letsencrypt.org/directory
```

```
    # 인증서 만료 또는 갱신 필요시 알람 email
```

```
    email: inky4832@gmail.com
```

```
    privateKeySecretRef:
```

```
      # 인증서를 발급받기 위해 ca와 통신시에 사용되는 Secret 이름
```

```
      name: my-issuer
```

```
  solvers:
```

```
    - http01:
```

```
      ingress:
```

```
        # 도메인의 소유권을 확인하기 위해 cert-manager는 nginx기반의 ingress controller를 활용해 검증한다는 의미
```

```
        class: nginx
```

15.5 EKS + https 적용

apiVersion: cert-manager.io/v1

kind: Certificate

metadata:

name: **app-inky4832-click-tls**

spec:

cert-manager가 아래 secretName에서 지정한 이름으로 Secret을 생성

secretName: **app-inky4832-click-tls**

duration: 2160h

#90day

renewBefore: 360h

#before 15day

issuerRef:

ClusterIssuer의 이름을 참조

name: my-issuer

kind: ClusterIssuer

commonName: **app.inky4832.click**

dnsNames:

추가적인 서브도메인을 추가하고 싶은 경우에 여기에 나열

- **app.inky4832.click**

15.5 EKS + https 적용

6. 로컬(윈도우)에 있는 deploy_AWS.yaml 수정하기

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: account-backend

spec:
  revisionHistoryLimit: 2
  replicas: 1

selector:
  matchLabels:
    app: account-backend
```

15.5 EKS + https 적용

```
template:
  metadata:
    labels:
      app: account-backend

  spec:
    containers:
      - name: account-backend
        image: 727646470302.dkr.ecr.ap-northeast-2.amazonaws.com/my-ecr:1.0
        imagePullPolicy: IfNotPresent

        ports:
          - containerPort: 8080

        resources:
          limits:
            cpu: "1"
            memory: "500Mi"
          requests:
            cpu: "0.5"
            memory: "250Mi"

        env:
          - name: DB_HOST
            valueFrom:
              secretKeyRef:
                name: my-app-secrets
                key: DB_HOST
```

15.5 EKS + https 적용

```
- name: DB_PORT
  valueFrom:
    secretKeyRef:
      name: my-app-secrets
      key: DB_PORT

- name: DB_NAME
  valueFrom:
    secretKeyRef:
      name: my-app-secrets
      key: DB_NAME

- name: DB_USERNAME
  valueFrom:
    secretKeyRef:
      name: my-app-secrets
      key: DB_USERNAME

- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: my-app-secrets
      key: DB_PASSWORD
```

```
livenessProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 8080
  initialDelaySeconds: 100
  periodSeconds: 20

readinessProbe:
  httpGet:
    path: /actuator/health/readiness
    port: 8080
  initialDelaySeconds: 100
  periodSeconds: 10
```

15.5 EKS + https 적용

```
---
apiVersion: v1
kind: Service
metadata:
  name: account-backend-service

spec:
  type: ClusterIP

  ports:
    - port: 80
      targetPort: 8080

  selector:
    app: account-backend
```


15.5 EKS + https 적용

7. 로컬(윈도우)에 있는 ingress_https_AWS.yaml 수정하기

```
kubectl apply -f ingress_https_AWS.yaml
```

apiVersion: networking.k8s.io/v1

kind: Ingress

metadata:

name: account-backend-ingress

annotations:

cert-manager.io/cluster-issuer: my-issuer

spec:

ingressClassName: nginx

tls:

- hosts:

- app.inky4832.click

secretName: app-inky4832-click-tls

rules:

- host: app.inky4832.click

http:

paths:

- path: /

pathType: Prefix

backend:

service:

name: account-backend-service

port:

number: 80

15.5 EKS + https 적용

8. 생성

```
kubectl apply -f db_secret_https_AWS.yaml  
kubectl apply -f redis_https_AWS.yaml  
kubectl apply -f ingress_https_AWS.yaml  
kubectl apply -f https_Cert_AWS.yaml  
kubectl apply -f deploy_https_AWS.yaml
```

16. 확인

```
kubectl get secret  
kubectl get deployment  
kubectl get pods  
kubectl get services  
kubectl get certificate  
kubectl get ingress
```

15.5 EKS + https 적용

회원가입및account생성

https://app.inky4832.click/api/signup-accounts

METHOD: **POST** | URL: **https://app.inky4832.click/api/signup-accounts** | length: 46 byte(s)

HEADERS: ☒ Content-Type: application/json

BODY:

```
{
  "username": "인경열2",
  "password": "1234",
  "email": "inky2@test.com",
  "mobileNumber": "01022228888"
}
```

Response

201

HEADERS: date: Sun, 12 Jul 2026 15:38:53 GMT -21m, content-type: application/json, vary: Origin, vary: Access-Control-Request-Method, vary: Access-Control-Request-Headers, x-content-type-opts: nosniff

BODY:

```
{
  "statusCode": "201",
  "statusMsg": "계정이 성공적으로 생성되었습니다."
}
```

length: 83 bytes

15.5 EKS + https 적용



15.6 CI/CD, Github Actions

15.6 CI/CD , Github Actions

1. Github action 파일 작성 .github/workflow/deploy.yaml

```
name: deploy account-backend
```

```
on:
```

```
  push:
```

```
    branches:
```

```
      - main
```

```
jobs:
```

```
  build-and-deploy:
```

```
    runs-on: ubuntu-latest
```

```
  steps:
```

```
    - name: Checkout GitHub
```

```
      uses: actions/checkout@v4
```

```
    - name: Install kubectl
```

```
      uses: azure/setup-kubectl@v4
```

```
      with:
```

```
        version: "v1.25.9"
```

```
- name: Configure AWS credentials
```

```
  uses: aws-actions/configure-aws-credentials@v4
```

```
  with:
```

```
    aws-access-key-id: ${ secrets.AWS_KEY }
```

```
    aws-secret-access-key: ${ secrets.AWS_SECRET }
```

```
    aws-region: ap-northeast-2
```

```
- name: Update EKS kubeconfig
```

```
  run: |
```

```
    aws eks update-kubeconfig \
```

```
      --name my-cluster \
```

```
      --region ap-northeast-2
```

15.6 CI/CD , Github Actions

```
- name: Login to Amazon ECR
  uses: aws-actions/amazon-ecr-login@v2

- name: Build and push Docker image
  env:
    REGISTRY: 727646470302.dkr.ecr.ap-northeast-2.amazonaws.com
    REPOSITORY: my-ecr
    IMAGE_TAG: ${ github.sha } # 1.0

  run: |
    docker build \
      -t $REGISTRY/$REPOSITORY:$IMAGE_TAG \
      -f ./Dockerfile \
      .

    docker push $REGISTRY/$REPOSITORY:$IMAGE_TAG

- name: Apply Kubernetes resources
  run: |
    kubectl apply -f "/k8s_AWS_https적용후/db_secret_https_AWS.yaml"
    kubectl apply -f "/k8s_AWS_https적용후/redis_https_AWS.yaml"
    kubectl apply -f "/k8s_AWS_https적용후/https_Cert_AWS.yaml"
    kubectl apply -f "/k8s_AWS_https적용후/deploy_https_AWS.yaml"
    kubectl apply -f "/k8s_AWS_https적용후/ingress_https_AWS.yaml"
```

15.6 CI/CD , Github Actions

```
- name: Update Deployment image
  env:
    REGISTRY: 727646470302.dkr.ecr.ap-northeast-2.amazonaws.com
    REPOSITORY: my-ecr
    IMAGE_TAG: ${ github.sha }

  run: |
    kubectl set image deployment/account-backend \
      account-backend=$REGISTRY/$REPOSITORY:$IMAGE_TAG

- name: Check Deployment rollout
  run: |
    kubectl rollout status deployment/account-backend \
      --timeout=300s

- name: Check Kubernetes resources
  run: |
    kubectl get deployment account-backend
    kubectl get pods -l app=account-backend
    kubectl get service account-backend-service
```


15.6 CI/CD , Github Actions

2. Github 에 저장소 생성

- 저장소명: **app-monolithic-cicd**

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).

Required fields are marked with an asterisk (*).

1 General

Owner *

 inky4832 ▾

Repository name *

app-monolithic-cicd

✓ app-monolithic-cicd is available.

app-monolithic-cicd

Great repository names are short and memorable. How about [crispy-potato](#)?

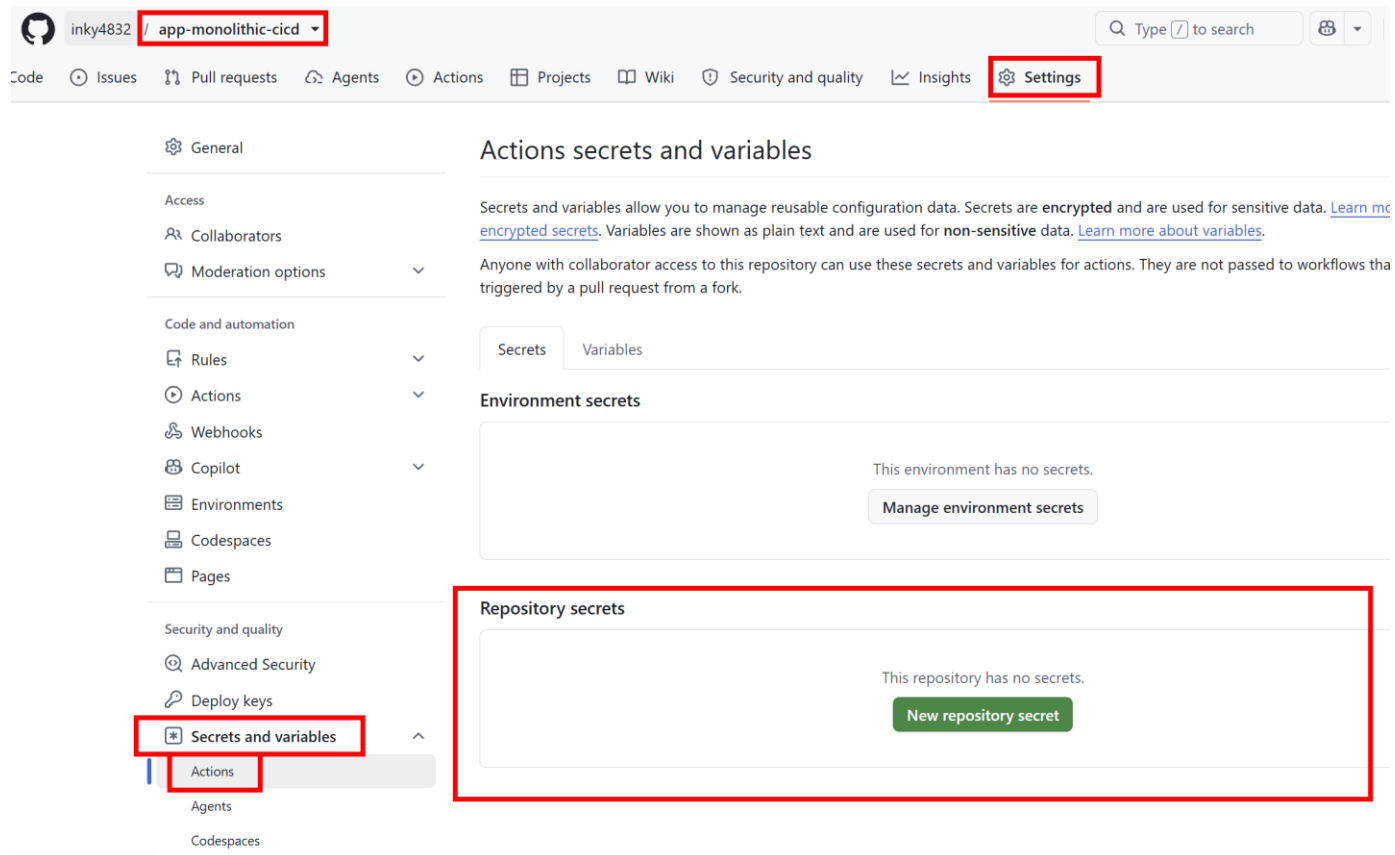
...or create a new repository on the command line

```
echo "# app-monolithic-cicd" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/inky4832/app-monolithic-cicd.git
git push -u origin main
```

15.6 CI/CD , Github Actions

3. Github 에 secret 설정

- app-monolithic-cicd > Settings > Secrets and variables > Actions > Repository secrets



15.6 CI/CD , Github Actions

Actions secrets / New secret

Name *

AWS_KEY

Secret *

AKIA2S2Y4ACPFTO33HPA

name: Configure AWS credentials
uses: aws-actions/configure-aws-credentials@v4
with:
aws-access-key-id: \${{ secrets.AWS_KEY }}
aws-secret-access-key: \${{ secrets.AWS_SECRET }}
aws-region: ap-northeast-2

Repository secrets

Add secret

Actions secrets / New secret

Name *

AWS_SECRET

Secret *

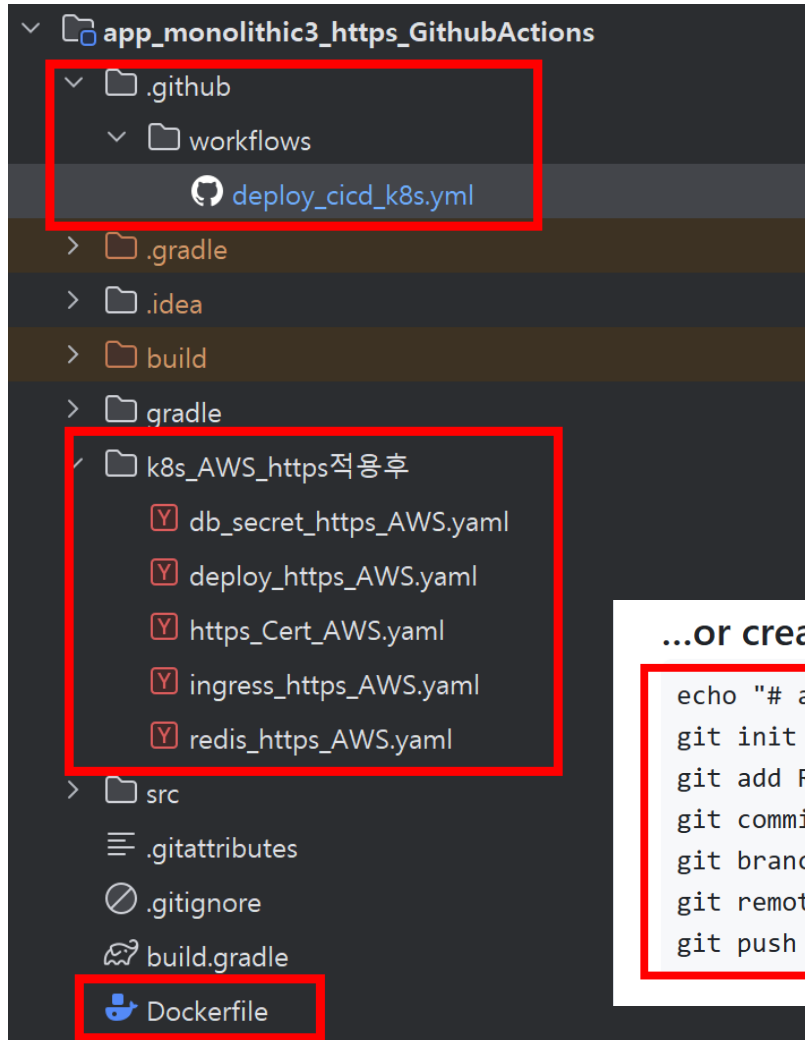
3cQmePNg4ID1BL+S0mp5gl6dl/N3+ptBTFzy/age

New repository secret

Name	Last updated
AWS_KEY	4 minutes ago
AWS_SECRET	now

15.6 CI/CD , Github Actions

4. SpringBoot 프로젝트 작성



...or create a new repository on the command line

```
echo "# app-monolithic-cicd" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/inky4832/app-monolithic-cicd.git
git push -u origin main
```

15.6 CI/CD , Github Actions

5. github actions 실행

The screenshot shows the GitHub Actions interface for the repository `inky4832 / app-monolithic-cicd`. The **Actions** tab is selected in the top navigation bar. On the left sidebar, the **workflows** section is active. The main content area displays **All workflows** and shows **1 workflow run**. A red box highlights the workflow run details for **first commit**, which is a **deploy account-backend #1** triggered by a commit push to the **main** branch. The commit hash is `050ca0e` and it was pushed by `inky4832`. A red text annotation **github에 Push 할때마다 동작됨** (Runs every time I push to GitHub) is placed over the workflow run details.

15.6 CI/CD , Github Actions

inky4832 / app-monolithic-cicd

Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

ons [New workflow](#)

workflows

oy account-backend

agement

Caches

Attestations

All workflows

Showing runs from all workflows

1 workflow run

녹색이면 배포 성공

✓ first commit

deploy account-backend #1: Commit [050ca0e](#) pushed by [inky4832](#)

main

15.6 CI/CD , Github Actions

Summary

All jobs

build-and-deploy

Run details

Usage

Workflow file

build-and-deploy

succeeded 2 minutes ago in 3m 17s

> ✓ Checkout GitHub

> ✓ Install kubectl

> ✓ Configure AWS credentials

> ✓ Update EKS kubeconfig

> ✓ Login to Amazon ECR

> ✓ Build and push Docker image

> ✓ Apply Kubernetes resources

> ✓ Update Deployment image

> ✓ Check Deployment rollout

✓ Check Kubernetes resources

1 ▶ Run kubectl get deployment account-backend

11	NAME	READY	UP-TO-DATE	AVAILABLE	AGE
12	account-backend	1/1	1	1	90m

13	NAME	READY	STATUS	RESTARTS	AGE
14	account-backend-c5fb697f6-wp51r	1/1	Running	0	108s

15	NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
16	account-backend-service	ClusterIP	172.20.232.180	<none>	80/TCP	90m

> ✓ Post Login to Amazon ECR

15.6 CI/CD , Github Actions

my-ecr

요약 | **이미지** | 수명 주기 정책 | 권한 | 리포지토리 태그

이미지 (7) 정보 ⓒ 삭제 URI 복사 세부 정보 스캔 푸시 명령 보기

🔍 활성 이미지 필터링

ECR에 push 성공

☐	이미지 태그 ▼	유형 ▼	생성 날짜: ▼	이미지 크기(MB) ▼	이미지 다이제스트	마지막으로 풀링된 시간: ▼
☐	050ca0e5867eac95c74d c842b4d44d7d64224ad d	Image	2026년 7월 13일, 23:50:50 (UTC+09)	140.66	sha256:17970bec...	2026년 7월 13일, 23:51:03 (UTC+09)

15.6 CI/CD , Github Actions

METHOD SCHEME :// HOST [":" PORT] [PATH ["?" QUERY]]

GET https://app.inky4832.click/health length: 33 byte(s) Send

▶ QUERY PARAMETERS

HEADERS [?] Form ▼ BODY [?]

+ Add header 🔗 Add authorization

XHR does not allow payloads for GET request.

Response

Cache Detected - Elapsed Time: 109ms

200

HEADERS [?] pretty ▼ BODY [?] raw ▼

date: Mon, 13 Jul 2026 14:57:19 GMT

content-type: text/plain; charset=UTF-8

health check !!!!

⬆ Top ⬇ Bottom 📄 2Request 📋 Copy 📄 Download

15.6 CI/CD , Github Actions

회원가입및account생성

Save

METHOD POST SCHEME :// HOST [":" PORT] [PATH ["?" QUERY]]

<https://app.inky4832.click/api/signup-accounts>

length: 46 byte(s)

QUERY PARAMETERS

HEADERS ②

Form

☒ Content-Type : application/json

+ Add header Add authorization

BODY ②

```

1 {
2   "username": "이순신",
3   "password": "1234",
4   "email": "lee@test.com",
5   "mobileNumber": "01022222222"
6 }

```

Response

Cache Detected - Elapsed

201

HEADERS ②

pretty

date: Mon, 13 Jul 2026 14:58:44 GMT

content-type: application/json

vary: Origin

vary: Access-Control-Request-Method

vary: Access-Control-Request-Headers

BODY ②

```

{
  statusCode: "201",
  statusMsg: "계정이 성공적으로 생성되었습니다."
}

```

Result Grid

Filter Rows:

Edit: Export/Import: Wrap Cell Content:

	created_at	customer_id	updated_at	created_by	email	mobile_number	password
▶	2026-07-13 14:58:44.156965	1	NULL	CARDS_MS	lee@test.com	01022222222	2a\$10\$C1gChqbfGxva3gGkTwZqu86xGANK9.
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

수고하셨습니다.